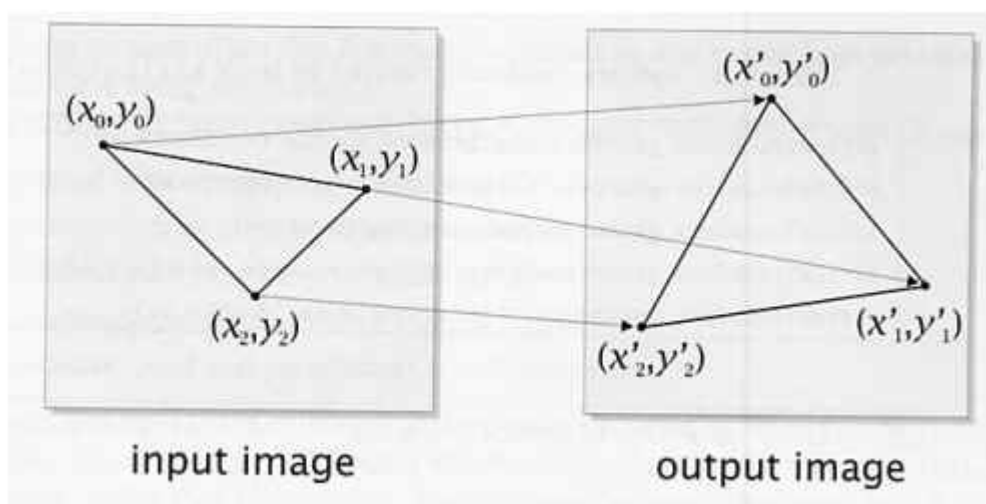


บทที่ 7

การขยายภาพ

7.1 Geometric Transformation

คือการเปลี่ยนแปลงภาพ เช่น หมุนภาพ, เลื่อนภาพ, ขยายหรือย่อภาพ, บิดภาพ เป็นต้น



รูปที่ 7-1 Geometric Transformation

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_0 & a_1 \\ b_0 & b_1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a_2 \\ b_2 \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_0 & a_1 & a_2 \\ b_0 & b_1 & b_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

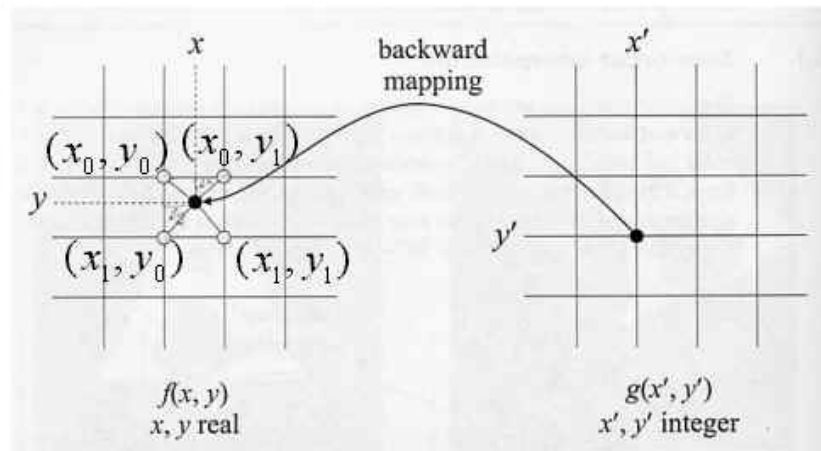
รูปที่ 7-2 ฟังก์ชัน Geometric Transformation

1) Rotation	 $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$
2) Translation (Shift)	 $\begin{aligned} x' &= x - \Delta x \\ y' &= y - \Delta y \end{aligned}$
3) Enlarge or reduce by scaling factor (S)	 $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \frac{1}{S} & 0 \\ 0 & \frac{1}{S} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$
4) Shear (บีค)	 $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a_0 & a_1 \\ b_0 & b_1 \end{bmatrix}^{-1} \begin{bmatrix} x \\ y \end{bmatrix}$

ตารางที่ 7-1 Geometric Transform Parameters (Backward Mapping)

7.2 Backward mapping

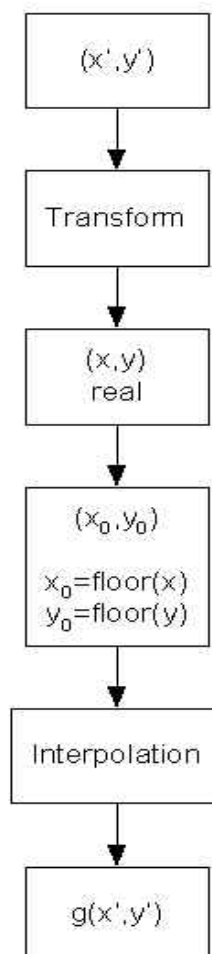
เป็นการมองจากผลลัพธ์ไปหาค่าเริ่มต้นจะทำให้ไม่เกิดปัญหา กรณี 2 จุดเมื่อผ่านฟังก์ชันแล้วได้จุดเดียวกัน หรือได้ค่าไม่ลงตัว



รูปที่ 7-3 Backward mapping

ฟังก์ชัน Transformation

$$g(x', y') = \text{interpolation}[f(x_0, y_0)]$$



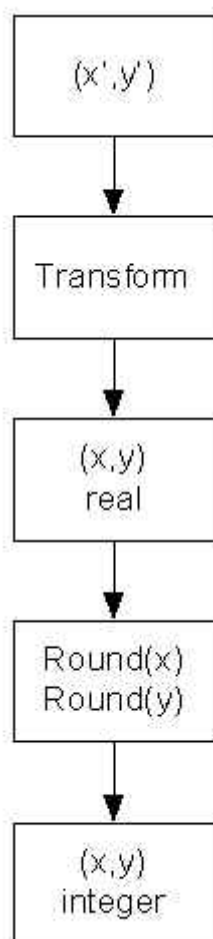
รูปที่ 7-4 Flow Chart Backward Mapping

7.3 การขยายภาพ โดยใช้ Zero-order Interpolation

Zero-order Interpolation เป็นกระบวนการประมวลผลทางรูปภาพ เรื่อง Geometric Transformation ใช้ในการเพิ่มหรือลดขนาดของภาพ ขึ้นกับค่า Scaling factor (S) > 1

Zero-order Interpolation เป็นวิธีที่เร็วกว่าการใช้ First-order Interpolation และ High-order Interpolation เพราะการคำนวณ มีความซับซ้อนน้อยกว่ามากทำให้ใช้เวลาน้อยในการคำนวณ ฟังก์ชัน Transformation

$$g(x',y') = f[\text{round}(x),\text{round}(y)]$$



รูปที่ 7-5 Flow Chart Zero-order Interpolation

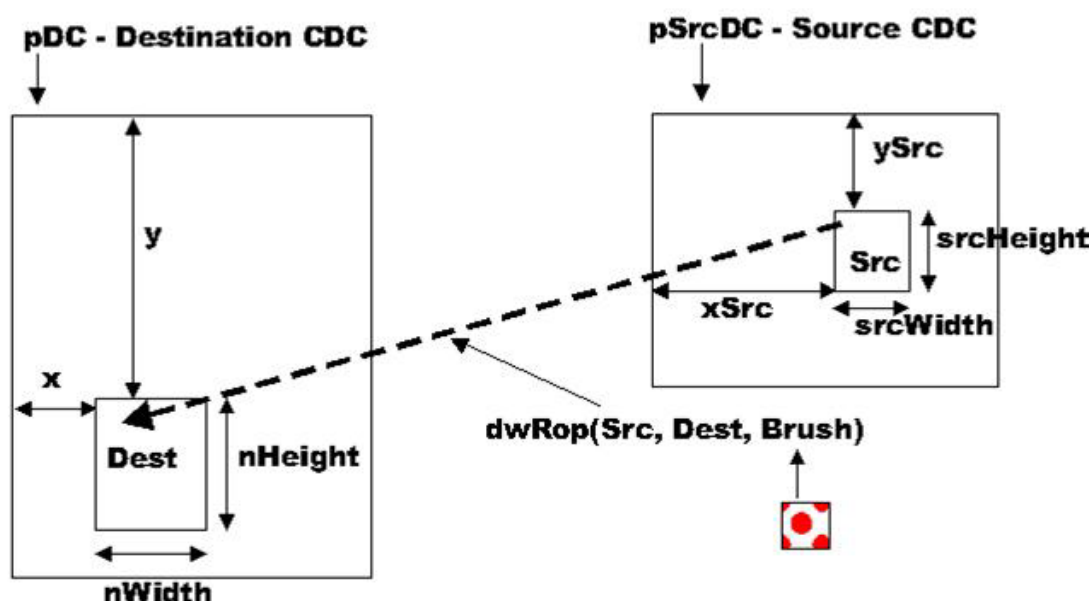
7.4 การขยายภาพ โดยใช้ฟังก์ชัน StretchBlt

เราสามารถขยายขนาดของภาพโดยใช้ฟังก์ชัน StretchBlt แทนซึ่งมีความรวดเร็วการที่เราเขียนฟังก์ชันการขยายภาพแบบทฤษฎี First Order เพราะเป็นการทำงานระหว่างหน่วยความจำ โดยมีขั้นตอนในการใช้งานดังต่อไปนี้

1. จะต้องประกาศส่วน Declaration ในตอนต้นของโปรแกรมก่อนใช้งาน

Declare Function StretchBlt Lib "gdi32" (ByVal hdc As Long, ByVal x As Long, ByVal y As Long, ByVal nWidth As Long, ByVal nHeight As Long, ByVal hSrcDC As Long, ByVal xSrc As Long, ByVal ySrc As Long, ByVal nSrcWidth As Long, ByVal nSrcHeight As Long, ByVal dwRop As Long) As Long

nRc = pDC->StretchBlt(x, y, nWidth, nHeight, pSrcDC, xSrc, ySrc, srcWidth, srcHeight, dwRop);



รูปที่ 7-6 แสดงการขยายภาพของฟังก์ชัน StretchBlt

2. จะต้องระบุตำแหน่งเริ่มต้นของภาพต้นทางและปลายทาง

ในการทำงานของฟังก์ชัน StretchBlt จะต้องระบุตำแหน่งภาพของภาพต้นทางว่าจะเริ่มที่ไหน และตำแหน่งภาพของภาพปลายทางว่าเริ่มที่ไหน เพื่อที่จะเป็นจุดเริ่มต้นของการเริ่มคัดลอกภาพ(copy) จากภาพต้นทางไปปลายทาง

3. ระบุขนาดของภาพต้นทางและปลายทาง

โดยที่เราจะต้องกำหนดขนาดของภาพต้นทางและปลายทางที่ต้องการ โดยเราสามารถระบุได้ว่าภาพต้นทางมีขนาดเท่านี้ ภาพปลายทางจะให้มีความเท่าไรก็ได้ โดยจะแสดงในรูปที่ 7-6

ตัวอย่างของการใช้ฟังก์ชัน StretchBlt

```
Dim hDC, hBMP As Long
hDC = CreateCompatibleDC(PictureB.hdc)
hBMP = CreateCompatibleBitmap(PictureB.hdc, 256, 192)
SelectObject hDC, hBMP

Dim MemDC, MemBMP As Long
MemDC = CreateCompatibleDC(PictureB.hdc)
MemBMP = CreateCompatibleBitmap(PictureB.hdc, 256, 192)
SelectObject MemDC, MemBMP

hDC = GetDC(ezVidCap1.hwnd)
StretchBlt MemDC, 0, 0, 256, 192, hDC, 0, 0, 64, 48, vbSrcCopy
```

จากตัวอย่างคือการขยายภาพเป็น 4 เท่าโดยที่จุดเริ่มต้นของภาพต้นทาง คือ (0,0) โดยขนาดของภาพต้นทางคือ กว้าง 64 x สูง 48

ในขณะที่ภาพปลายทางจะเริ่มส่งรูปมาที่ตำแหน่ง (0,0) โดยที่ขนาดของภาพปลายทางมีขนาดคือ กว้าง 256 x สูง 192 ซึ่งมีขนาดเป็น 4 เท่าของภาพต้นทาง